05 - SDLC & Project Management Process

LIONGATE SARL - Engineering Standards

Version 1.0 - Initial release

Table of Contents

Overview

What is This Document?

This document defines the full Software Development Lifecycle (SDLC) for all projects at LIONGATE SARL. It is the source of truth for how software is built, reviewed, deployed, and maintained.

SDLC Model

LIONGATE SARL uses a **hybrid Agile model**:

- **Sprint-based iteration** (2-week sprints) for active development phases
- **Waterfall checkpoints** for project kickoff, architecture, and go-live decisions
- **Continuous delivery** for production deployments once the system is live

Project Lifecycle Summary

```
Discovery → Requirements → Estimation → Architecture
     ↓
Development (Sprints) → Code Review → Testing
     ↓
Staging → Approval → Deployment
     ↓
Monitoring → Maintenance → Change Management → Support
```

Team Roles & Responsibilities

Role Definitions

Role	Abbreviation	Responsibilities
Product Owner	PO	Owns product vision, accepts/rejects stories, prioritizes backlog, represents business/client
Project Manager	PM	Plans sprints, tracks milestones, manages risks, coordinates team, client communication

Role	Abbreviation	Responsibilities
Tech Lead	TL	Architecture decisions, code quality, technical direction, final technical reviewer
Backend Engineer	BE	Implements APIs, services, database logic, integrations
Frontend Engineer	FE	Implements UI, state management, API integration
DevOps Engineer	DO	Infrastructure, CI/CD, deployments, monitoring, security
QA Engineer	QA	Test strategy, test cases, manual and automated testing
Designer	DES	UI/UX design, wireframes, design system, prototypes
Support Engineer	SE	Incident triage, bug reports, client-facing technical support

RACI Matrix (per phase)

Phase	PO	PM	TL	BE	FE	DO	QA	DES
Discovery	R/A	C	C	I	I	I	I	C
Requirements	R/A	C	C	C	C	I	C	C
Estimation	I	A	R	C	C	C	C	C
Architecture	I	I	R/A	C	C	C	I	I
Development	I	A	R	R	R	C	I	I
Code Review	I	I	A	R	R	I	I	I
Testing	C	A	C	C	C	I	R	I
Staging	A	R	C	C	C	C	R	I
Deployment	I	A	C	C	C	R	I	I
Monitoring	I	I	C	C	C	R/A	I	I
Support	C	A	C	C	C	C	C	I

R = Responsible, A = Accountable, C = Consulted, I = Informed

Project Lifecycle Phases

Phase Duration Estimates

Phase	Typical duration
Discovery	1–2 weeks
Requirements	1–3 weeks
Estimation	3–5 days
Architecture	1–2 weeks
Development (sprints)	4–16 weeks (project-dependent)
Testing (per sprint)	Continuous + 1 dedicated QA sprint
Staging / UAT	1–2 weeks
Deployment	1–2 days
Maintenance	Ongoing (monthly retainer or ad hoc)

Phase 1: Discovery

Purpose

Understand the problem, align on scope, identify risks before committing resources.

Activities

Activity	Owner	Output
Initial stakeholder meeting	PM	Meeting notes
Business problem statement	PO	Problem brief

Activity	Owner	Output
Existing system analysis	TL	Technical assessment
Competitor / market review	PO	Reference document
Feasibility check	TL + DO	Risk log
Project scope definition (high-level)	PM + PO	Scope document
Budget range discussion	PM + PO	Budget brief

Discovery Deliverables

```
discovery/
├── problem-brief.md
├── scope-summary.md
├── technical-feasibility.md
├── initial-risk-log.md
└── meeting-notes/
    └── kickoff-YYYYMMDD.md
```

Exit Criteria for Discovery

- ☐ Problem clearly articulated in writing
- ☐ High-level scope agreed by client/stakeholder
- ☐ Initial feasibility confirmed by TL
- ☐ Budget range acknowledged
- ☐ Go/No-Go decision made → if Go, proceed to Requirements

Phase 2: Requirements Gathering

Purpose

Translate business needs into clear, testable requirements that engineers can implement.

Activities

Activity	Owner	Output
Stakeholder interviews	PM + PO	Interview notes
User stories writing	PO + PM	Story backlog
UI/UX wireframes	DES	Wireframes / mockups
Data model draft	TL + BE	ER diagram
Integration mapping	TL	Integration list
Non-functional requirements	TL	NFR document
Acceptance criteria per story	PO + QA	Story criteria

User Story Format

```
Title: [Verb] [object] as [role]

As a [user role],
I want to [action / goal],
So that [business value].

Acceptance Criteria:
  GIVEN [precondition]
  WHEN [action]
  THEN [expected outcome]

  GIVEN [precondition]
  WHEN [action]
  THEN [expected outcome]

Definition of Done (standard – see Section 19)
Estimation: [story points]
Priority: [Must / Should / Could / Won't]
```

Requirements Document Structure

```
requirements/
├── functional/
│   ├── user-management.md
│   ├── authentication.md
│   └── [feature].md
├── non-functional/
│   ├── performance.md    ← e.g., "API must respond in < 300ms at P95"
│   ├── security.md
│   ├── scalability.md
│   └── availability.md
├── data-model/
│   └── erd.drawio / erd.png
└── wireframes/
    └── [screen-name].fig / .png
```

Non-Functional Requirements (NFR) Template

Category	Requirement	Measurement
Performance	API P95 < 300ms	Load test: 100 concurrent users
Availability	99.5% uptime	Monthly window
Scalability	Support 500 concurrent users	Load test
Security	OWASP Top 10 compliance	Security audit
Backup	RPO < 24h, RTO < 2h	DR test
Data retention	Logs 90 days, audit 1 year	Policy

Phase 3: Estimation

Purpose

Produce a realistic time and cost estimate for the project.

Estimation Method: Story Points + Reference Velocity

```
Story points scale: 1, 2, 3, 5, 8, 13, 21

Reference:
  1 point  = trivial change, < 2 hours
  2 points = simple feature, 2–4 hours
  3 points = medium feature, 4–8 hours
  5 points = complex feature, 1–2 days
  8 points = very complex, 2–4 days (consider splitting)
  13 points = should be split
  21 points = must be split

Velocity reference (per 2-week sprint, 1 engineer):
  Average team velocity: 20–30 story points per engineer per sprint
  Apply 30% buffer for unknowns

Total hours = (total_points / velocity_per_day) × working_days
```

Estimation Document Template

```
# Project Estimation: [Project Name]

Date: [Date]
Estimators: [Names]

## Assumptions

- [List all assumptions]

## Out of scope

- [List explicitly excluded items]

## Story Point Summary

| Epic            | Stories | Points  |
| --------------- | ------- | ------- |
| Auth            | 5       | 18      |
| User Management | 8       | 32      |
| Dashboard       | 4       | 15      |
| API Integration | 6       | 28      |
| DevOps / Infra  | 3       | 12      |
| **Total**       | **26**  | **105** |

## Time Estimate

Team: 2 BE, 1 FE, 0.5 DevOps
Velocity: 25 pts/sprint/engineer
Sprints needed: 105 / (25 × 3) = ~1.4 sprints → 2 sprints with buffer
Duration: ~4 weeks + 1 QA sprint = 6 weeks

## Cost Estimate (if billing)

Rate: [Daily rate]
Duration: 6 weeks × 5 days × [daily rate] × [engineers] = XAF **\_\_**

## Risk buffer: +20%

## Final estimate: [Final figure]
```

Phase 4: Architecture

Purpose

Define the technical system design before coding begins.

Architecture Deliverables

Deliverable	Owner	Format
Architecture diagram	TL	Draw.io / Mermaid
Technology stack decision	TL	ADR
Data model (final)	TL + BE	SQL schema
API design	TL + BE	OpenAPI spec
Infrastructure plan	DO	Terraform plan
Security design	TL	Threat model
Deployment architecture	DO	Architecture diagram

Architecture Decision Record (ADR)

Every significant technical decision must be documented as an ADR.

```
# ADR-001: Use PostgreSQL as primary database

**Date:** 2024-01-15
**Status:** Accepted
**Deciders:** Tech Lead, Backend Engineer

## Context

We need to choose a database for the project. Options considered: PostgreSQL, MySQL, Mo

## Decision

We will use PostgreSQL 16.

## Rationale

- JSONB support for flexible data structures
- Superior transaction support and ACID compliance
- Strong ecosystem (TypeORM, Prisma, pg_dump)
- LIONGATE standard (reduces operational overhead)
- Better performance for complex joins vs MySQL

## Consequences

- Team must be familiar with PostgreSQL syntax
- Migration from MySQL (if client has existing) requires planning
- Managed DB option available on Hetzner for SLA requirements

## Alternatives Rejected

- MySQL: inferior JSONB and window function support
- MongoDB: no ACID transactions, overkill flexibility for this use case


docs/
└── adr/
    ├── ADR-001-database-postgresql.md
    ├── ADR-002-backend-nestjs.md
    └── ADR-003-frontend-nextjs.md
```

Phase 5: Development

Sprint Structure (2 weeks)

```
Monday Week 1:     Sprint Planning
Daily (async):     Standup (written in Slack/Teams)
Tuesday Week 2:    Code freeze for sprint (features)
Wednesday Week 2:  Bug fixing and QA
Thursday Week 2:   Sprint Review + Demo
Friday Week 2:     Sprint Retrospective + Next Sprint Prep
```

Sprint Planning Agenda

1. Review sprint goal (5 min)
2. Review backlog — PO presents top items (15 min)
3. Engineers estimate any unpointed stories (15 min)
4. Agree on sprint scope based on velocity (10 min)
5. Assign stories to individuals (10 min)
6. Identify dependencies and blockers (5 min)

Total: ~60 min

Daily Standup Format (async, written)

```
**[Name] — [Date]**

Yesterday:
  - Completed: [what was done]

Today:
  - Working on: [story/task reference]

Blockers:
  - [None / Describe blocker]
```

Development Workflow (per story)

1. Pick story from sprint backlog
2. Create feature branch: feature/PROJ-123-short-description
3. Implement
4. Write/update tests
5. Run linter and tests locally (make test passes)
6. Open PR against develop
7. Link PR to story in issue tracker
8. Pass CI checks
9. Request review
10. Address review comments
11. Merge (squash merge)
12. Story moves to "In Review" → "Done" after QA approval

Coding Standards in Development

- Commit messages follow Conventional Commits
- No console.log / print statements in production code
- No TODO comments without a linked issue
- Functions > 50 lines should be split
- Cyclomatic complexity ≤ 10 per function

Phase 6: Code Review

Review Process

```
Author → Opens PR → Assigns reviewer(s)
              ↓
Reviewer → Reviews within 24h (business hours)
              ↓
         → Approved: Author merges
         → Changes requested: Author addresses, re-requests review
              ↓
         3rd round: Tech Lead resolves dispute
```

Reviewer Checklist

Functionality

- [ ] Code does what the story requires
- [ ] Edge cases handled
- [ ] Error handling present and appropriate

Quality

- [ ] No unnecessary complexity
- [ ] Functions are named clearly
- [ ] No dead code
- [ ] DRY (no duplicated logic)

Tests

- [ ] New code has tests
- [ ] Tests actually test behavior, not implementation
- [ ] Coverage acceptable

Security

- [ ] No hardcoded credentials
- [ ] Input validated
- [ ] RBAC applied to new endpoints
- [ ] No sensitive data logged

Database

- [ ] Migration included
- [ ] Indexes on new foreign keys / filtered columns
- [ ] No N+1 queries

Documentation

- [ ] API endpoints documented in Swagger
- [ ] Complex logic has inline comments
- [ ] README updated if setup changed

Review Communication Norms

```
Prefix review comments with severity:
  [blocking]  - Must be fixed before merge
  [nit]       - Minor, author's discretion
  [question]  - Seeking understanding, not requesting change
  [suggest]   - Optional improvement

Example:
  [blocking] This query will cause an N+1. Please use eager loading.
  [nit] Variable name could be more descriptive.
  [question] Why did you choose this approach over X?
```

Phase 7: Testing

Testing Strategy Per Project

Test type	Who	When	Mandatory
Unit tests	Developer	During development	Yes
Integration tests	Developer	During development	Yes
API contract tests	QA + Dev	Sprint review	For client projects
E2E tests	QA	After staging deploy	For client projects
Manual exploratory	QA	Sprint end	Yes
Regression	QA	Before production deploy	Yes
Performance / load	DevOps + QA	Pre-launch	For enterprise
Security scan	DevOps	In CI pipeline	Yes
UAT	Client / PO	Staging	Yes

Bug Severity Classification

Severity	Definition	Response time
P0 - Critical	Production down, data loss	Fix within 4 hours
P1 - High	Core feature broken, no workaround	Fix within 24 hours
P2 - Medium	Feature broken, workaround exists	Next sprint
P3 - Low	UI issue, minor behavior	Backlog

Bug Report Template

```
**Bug ID:** BUG-XXX
**Title:** [Concise description]
**Severity:** P0 / P1 / P2 / P3
**Reported by:** [Name]
**Date:** [Date]
**Environment:** Staging / Production

## Steps to Reproduce

1. Log in as [role]
2. Navigate to [page]
3. [Action]

## Expected Result

[What should happen]

## Actual Result

[What happens instead]

## Evidence

[Screenshots, logs, video]

## Notes

[Browser, OS, version, any additional context]
```

Phase 8: Staging

Purpose

Final validation in an environment identical to production before going live.

Staging Environment Rules

- Staging uses **anonymized production data** or realistic seed data
- Staging is reset and redeployed at the start of each UAT cycle
- Staging URL format: `staging.projectname.com` or `projectname-staging.liongate.com`
- Staging access is restricted to team + client (no public access)

UAT Process

1. Deploy to staging
2. Notify client/PO: "Staging ready for testing"
3. Share: staging URL, test user credentials, test scenarios list
4. UAT window: 3–5 business days
5. Client logs issues in issue tracker (Jira or GitHub Issues)
6. Team triages issues:
   - P0/P1: fix immediately, redeploy staging
   - P2/P3: decide in meeting
7. Client signs off: written confirmation required
8. Proceed to production deployment planning

UAT Sign-Off Template

Project: [Name]
Staging version: v[X.Y.Z]
UAT Start: [Date]
UAT End: [Date]

Tested scenarios: [X / Y completed]
Issues found: [N total — N P0, N P1, N P2, N P3]
Issues resolved: [N]
Open issues (accepted for post-launch): [N]

Sign-off: _____
Role: Product Owner / Client Representative
Date: _____

Phase 9: Deployment

Production Deployment Process

Pre-deployment:

1. Final staging validation ✓
2. UAT sign-off received ✓
3. Database backup taken ✓
4. Rollback plan documented ✓
5. Maintenance window scheduled and communicated ✓
6. Team on standby ✓

Deployment:

1. Enable maintenance page (if needed)
2. Take final DB backup
3. Run migration (staging first if schema change)
4. Deploy new Docker image
5. Run smoke tests
6. Remove maintenance page
7. Monitor for 30 minutes

Post-deployment:

8. Verify all critical paths in production
9. Notify stakeholders: "Deployment complete"
10. Monitor logs and metrics for 2 hours
11. Document deployment in changelog

Rollback Plan

```
Trigger for rollback:
  - P0 bug discovered in production
  - Health check failing
  - Error rate > 5%

Rollback steps:
  1. Notify team immediately
  2. Roll back Docker image to previous tag:
     docker compose pull image:previous-tag
     docker compose up -d --no-deps app
  3. If migration was run: restore pre-deployment DB backup
  4. Verify rollback succeeded
  5. Notify stakeholders of rollback and estimated resolution
```

Phase 10: Monitoring & Maintenance

Ongoing Monitoring Activities

Activity	Frequency	Owner
Review Grafana dashboards	Daily	DO
Check error logs	Daily	BE
Review backup logs	Daily	DO
Database performance review	Weekly	BE + DO
SSL cert expiry check	Monthly	DO
Dependency security scan	Weekly (CI)	DO
Infrastructure cost review	Monthly	DO + PM
Uptime report to client	Monthly	PM

Maintenance Types

Type	Description	Frequency
Security patches	OS + dependency updates	Monthly
Odoo/framework upgrades	Minor version updates	Quarterly
Database maintenance	VACUUM, REINDEX, statistics	Monthly
Log cleanup	Archive/delete old logs	Monthly
Backup testing	Restore test on staging	Monthly

Phase 11: Change Management

Change Request Process

```
1. Request received (from client or internal)
   ↓
2. PM logs as a Change Request (CR)
   ↓
3. TL assesses technical impact:
   – Complexity
   – Risk
   – Dependencies
   – Effort estimate
   ↓
4. PO assesses business impact:
   – Priority vs current backlog
   – Value
   ↓
5. If approved:
   – Add to backlog (next sprint or dedicated sprint)
   – Update project timeline if needed
   – Client notified of impact on cost/timeline
   ↓
6. Implement via standard development process
```

Change Request Template

```
**CR-ID:** CR-XXX
**Title:** [Short description]
**Requested by:** [Name / Client]
**Date:** [Date]

## Description

[What is being requested]

## Business Justification

[Why this is needed]

## Technical Assessment

Effort: [X story points / days]
Risk: Low / Medium / High
Impact on existing features: [None / Describe]
Dependencies: [None / List]

## Decision

[ ] Approved – Add to sprint [XX]
[ ] Deferred – Target sprint [XX]
[ ] Rejected – Reason: [Reason]

**Decided by:** [TL / PM / PO]
**Date:** [Date]
```

Phase 12: Support

Support Tiers

Tier	Response	Scope
T1 - Monitoring	Automated alert	System health, uptime

Tier	Response	Scope
T2 - Bug triage	Within 24h	Bug classification, workarounds
T3 - Engineering fix	Based on severity	Code-level resolution
T4 - Architecture	Scheduled	Deep issues, performance, scaling

Support Model Options

Model	Description	When to use
Ad hoc	Client reports issues, team responds	Small clients, low traffic
Monthly retainer	Dedicated hours per month	Active products
SLA-based	Defined response times per severity	Enterprise clients

Work Item Taxonomy

Work Item Hierarchy

```
Initiative (Strategic goal)
└── Epic (Large feature area, 1–3 months)
    └── Story (User-facing feature, 1–5 days)
        └── Task (Technical step, < 1 day)
            └── Subtask (optional, for breakdown)

Bug (Defect, cross-cutting)
Incident (Production event)
Change Request (Scope change)
Tech Debt (Internal improvement)
```

Work Item Naming Conventions

```
Epic:       [EPIC] User Management
Story:      [BE] User can reset password via email link
Task:       [FE] Implement password reset form component
Bug:        [BUG-P1] Password reset email not sent when email has uppercase
Incident:   [INC-001] Production DB connection pool exhausted - 2024-01-15
Tech Debt:  [TD] Refactor UserService to use repository pattern
```

Work Item Statuses

```
Backlog → Ready → In Progress → In Review → Testing → Staging → Done

Backlog:     Created, not yet groomed or estimated
Ready:       Groomed, estimated, acceptance criteria written
In Progress: Developer is actively working
In Review:   PR open, awaiting code review
Testing:     QA is testing
Staging:     Deployed to staging, awaiting UAT
Done:        Accepted by PO, merged, deployed to production
```

Estimation Framework

Estimation Rules

1. Estimates are done by the people who will do the work
2. Product Owner does NOT estimate (they define, engineers estimate)
3. Never estimate under pressure - "I'll estimate when I understand it"
4. If a story cannot be estimated (too vague), return to PO for clarification
5. Spike tasks (research) are time-boxed, not pointed (e.g., "2-day spike")
6. Infrastructure setup stories are estimated separately from feature stories

Three-Point Estimation (for uncertain tasks)

```
For risky or unclear stories:

Optimistic (O): Best case scenario
Most likely (M): Expected scenario
Pessimistic (P): Worst case scenario

Expected = (O + 4M + P) / 6
Standard deviation = (P − O) / 6

Example:
  O = 3 days, M = 5 days, P = 10 days
  Expected = (3 + 20 + 10) / 6 = 5.5 days
  Buffer = 5.5 + 6 = 2 days
  Final estimate: 7.5 days (round to 8)
```

Prioritization Framework

MoSCoW Method

Priority	Label	Definition
Must	M	Non-negotiable for launch
Should	S	High value, included if possible
Could	C	Nice to have, included if time allows
Won't	W	Excluded from current scope, documented

RICE Scoring (for competing priorities)

```
RICE Score = (Reach × Impact × Confidence) / Effort

Reach:      Number of users affected (per sprint)
Impact:     0.25 (minimal) / 0.5 (low) / 1 (medium) / 2 (high) / 3 (massive)
Confidence: 50% / 80% / 100% (how sure are we about above estimates?)
Effort:     Person-weeks

Higher score = higher priority
```

Definition of Done

Story-Level Definition of Done

A story is **Done** when ALL of the following are true:

- [ ] Code implemented and matches acceptance criteria
- [ ] Unit tests written and passing (coverage meets threshold)
- [ ] Integration tests updated/added if needed
- [ ] PR opened, reviewed, and approved
- [ ] CI pipeline passes (lint, tests, build, security scan)
- [ ] Migrations included (if DB changes)
- [ ] API documented in Swagger (if new endpoints)
- [ ] Code merged to `develop` branch
- [ ] Deployed to staging
- [ ] QA tested and signed off
- [ ] No P0/P1 bugs introduced
- [ ] PO accepted the story

Sprint-Level Definition of Done

A sprint is **Done** when ALL of the following are true:

- [ ] All committed stories meet story-level DoD
- [ ] No unresolved P0 or P1 bugs
- [ ] Sprint demo completed

- [ ] Retrospective completed
- [ ] Next sprint backlog groomed
- [ ] Changelog updated

Technical Debt Management

What Counts as Technical Debt

Type	Example
Code debt	Duplicated logic, complex function needing refactor
Test debt	Missing test coverage for critical paths
Architecture debt	Monolith needing decomposition
Dependency debt	Outdated packages with known vulnerabilities
Documentation debt	Missing ADRs, outdated README
Infrastructure debt	Unautomated manual processes

Technical Debt Process

```
1. Any team member can log a tech debt item in the backlog
2. Format: [TD] Short description
3. Tech Lead reviews monthly — tags: Critical / High / Low
4. Allocation rule:
   - 10–15% of each sprint capacity reserved for tech debt
   - Critical debt escalated to immediate sprint
5. Tech debt items tracked in dedicated backlog column
6. Quarterly tech debt review: TL + PM + stakeholder
```

Technical Debt Register (template)

ID	Description	Type	Impact	Effort	Priority	Sprint
TD-001	Extract config service from AppModule	Code	Medium	3 pts	High	Sprint 8

ID	Description	Type	Impact	Effort	Priority	Sprint
TD-002	Add index on orders.user_id	DB	High	1 pt	Critical	Sprint 7

Decision Documentation

Architecture Decision Records (ADRs)

See Phase 4 for ADR format. ADRs are stored in `docs/adr/` and versioned in Git.

Meeting Decisions

All decisions made in meetings must be:

1. Documented in meeting notes within 24 hours
2. Shared to the relevant team channel
3. Assigned an owner and due date if actionable

Decision Log Template

```
## Decision Log – [Project Name]

| Date       | Decision                    | Context          | Made by | Status |
| ---------- | --------------------------- | ---------------- | ------- | ------ |
| 2024-01-15 | Use PostgreSQL              | Standard stack   | TL      | Final  |
| 2024-01-20 | Defer real-time feature to v2 | Scope management | PO + PM | Final  |
```

Release Management

Release Types

Type	Trigger	Process
Feature release	End of milestone	Full staging + UAT + deploy

Type	Trigger	Process
Patch release	Bug fixes	Abbreviated UAT + deploy
Hotfix release	P0 production bug	Emergency: fix → test → deploy
Security release	CVE / vulnerability	Priority override, immediate

Release Checklist

- [ ] CHANGELOG.md updated with version and changes
- [ ] Version tag created (`v1.3.0`)
- [ ] GitHub Release created with notes
- [ ] Staging fully validated
- [ ] UAT sign-off (for feature releases)
- [ ] Backup taken before deployment
- [ ] Team on standby during deployment
- [ ] Post-deployment smoke test passed
- [ ] Stakeholders notified

CHANGELOG Format

```
# Changelog

## [1.3.0] - 2024-02-01

### Added

- User export to CSV
- Dashboard widget for active sessions

### Changed

- Improved pagination performance on user list

### Fixed

- Password reset email not sent for uppercase emails (BUG-042)

## [1.2.1] - 2024-01-20

### Fixed

- Order total calculation error for multi-currency (BUG-039)
```

Incident Management

Incident Severity Levels

Level	Description	Response
SEV1	Complete outage, all users affected	Immediate - all hands
SEV2	Major feature broken, >50% users	Within 1 hour
SEV3	Minor feature broken, <20% users	Within 4 hours
SEV4	Cosmetic or minor degradation	Next business day

Incident Response Process

Detection (monitoring alert or user report)
    ↓
1. DECLARE: Create incident channel/thread immediately
2. ASSIGN: Incident commander (IC) assigned (usually TL or SE)
3. INVESTIGATE: IC + BE diagnose root cause
4. COMMUNICATE: PM notifies stakeholders within 15 min (for SEV1/2)
5. MITIGATE: Apply fix or rollback to restore service
6. VERIFY: Confirm service restored, monitoring clean
7. CLOSE: Announce resolution to stakeholders
8. POST-MORTEM: Within 48 hours (blameless)

Incident Post-Mortem Template

```
# Post-Mortem: [Incident title]

**Date:** [Date]
**Severity:** SEV[1/2/3]
**Duration:** [Start] → [End] ([X] hours)
**Impact:** [N users affected / [X]% of traffic / [service degraded]]
**IC:** [Name]

## Timeline

- 14:32 – Alert fired: high 5xx rate on /api/v1/orders
- 14:35 – IC declared incident
- 14:40 – Root cause identified: DB migration locked table
- 15:00 – Fix deployed, service restored

## Root Cause

[Clear, factual description]

## Contributing Factors

[What made this possible or worse]

## Resolution

[What was done to fix it]

## Action Items

| Action                              | Owner  | Due Date |
| ----------------------------------- | ------ | -------- |
| Add pre-deployment migration check | DevOps | Sprint 9 |
| Add alert for table lock duration  | DevOps | Sprint 9 |

## What Went Well

- Team mobilized quickly
- Rollback plan was ready

## What Could Be Improved
```

- Alert threshold was too high (5 min delay)
- Migration should have been tested on a prod-sized dataset

Document owner: LIONGATE SARL Engineering & Operations Team
Last updated: See Git history
Next review: Quarterly